

Achim Schroll
20030325

Numerical Analysis for E3

Solutions for Homework assignment 1

1. A simple circuit.

Task 1a):

Denote the the tensions and the current V_{C_1}, V_{C_2} and $i_L(t)$ as $y_1 \dots y_3$, respectively. The right hand side equations can be written in matrix form as

$$\dot{y} = \begin{pmatrix} \dot{y}_1 \\ \dot{y}_2 \\ \dot{y}_3 \end{pmatrix} = \begin{pmatrix} -\frac{y(3)}{C_1} \\ -\frac{y(3)}{C_2} - \frac{y(2)}{RC_2} \\ \frac{y(1)+y(2)}{L} \end{pmatrix}. \quad (1)$$

The corresponding MATLAB code will be (function LCCR.m):

```
function y=LCCR(t,x)
c1=1e-5;
c2=1e-5;
L=1e-5;
R=0.2e1;

vc1=x(1);
vc2=x(2);
il=x(3);

y(1)=(-il)/c1;
y(2)=(-il-vc2/R)/c2;
y(3)=(vc1+vc2)/L;
y=y(:);
```

1.b): For the initial conditions we have that $V_{C_1} = 1, V_{C_2}(0) = 1$ and $i_L(0) = 0$. The MATLAB algorithm becomes

```
ode45('LCCR',[0 1e-3],[1 1 0]);  
title('A simple circuit example');  
xlabel('time');  
legend('V_{C_1}','V_{C_2}','i_L');
```

The first argument of the function `ode45` is the name of the right hand side equation, the second is the time interval $([0, 10^{-3}])$ and the third is the initial values vector.

The solution curves in the figure 1 can easily be separated by examining the initial and final values.

Why just the command `ode45`? if you looked at the help text of the file `ode45`, you surely observed that there were several other files with name starting with 'ode' and the some numbers indicating the type of integrator. Why not using another integrator instead? We will try to answer this question in the course when taking about Initial Value Problems, but in figure 2 we compared two simulations in the time interval $[0, 10^2]$, the first one done with `ode15s` and the second one with `ode45`. We observe in the first case that even having a dissipative element as a resistance R , the amplitudes of the tensions and current are increasing! (The total energy of the system is not at all being dissipated but increasing) This is related to the colocation of the poles and zeros of the impedance we are simulating. We will discuss these facts later on the course, but you observe the effect already now.

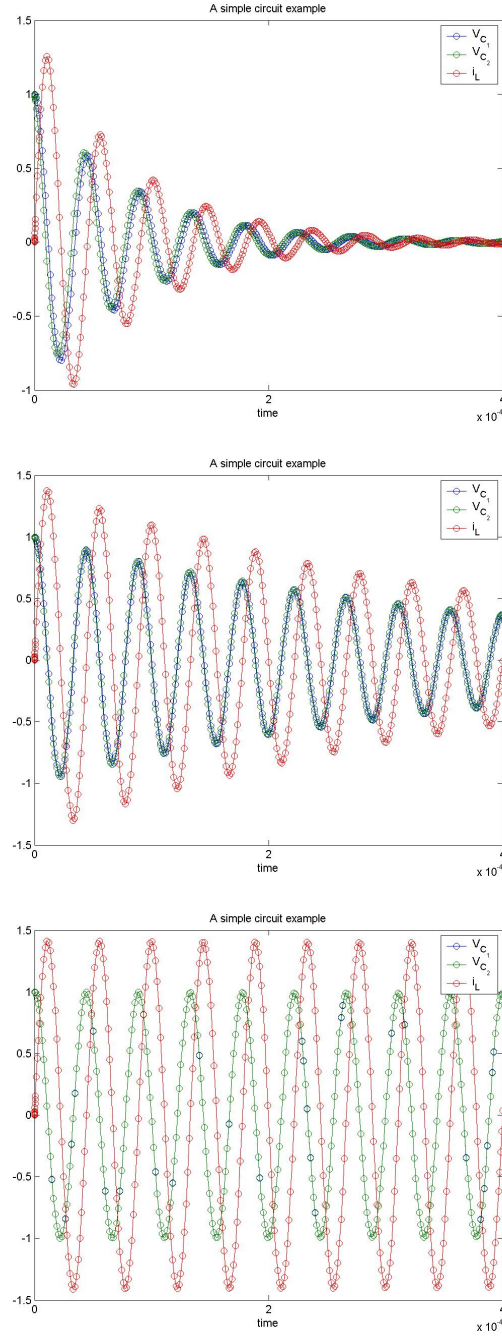


Figure 1: Results of the simulations for $R = 2, 10$ and 10000 ohms

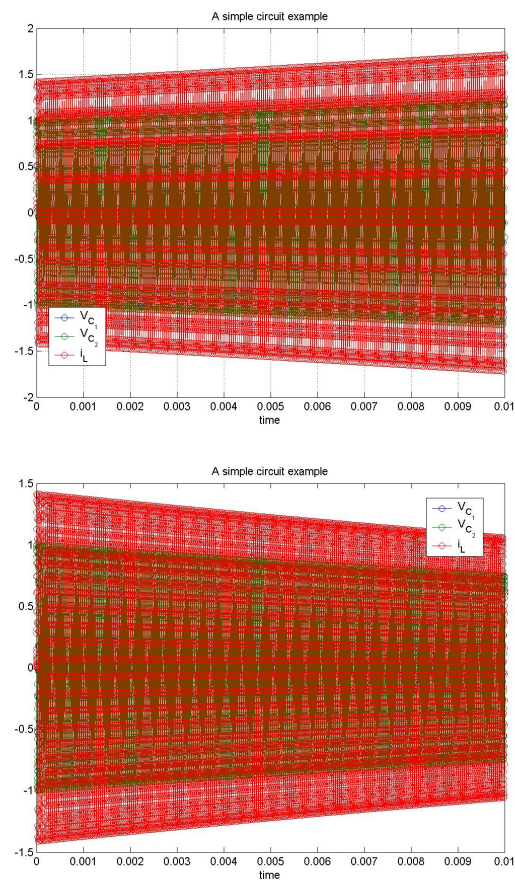


Figure 2: A comparison between `ode45` and `ode15s`.

2. Interpolation of tabulated data.

Task 2a):

The interpolation polynomial using Lagrange basis is in this case

$$p_2(t) = y_0 \cdot \frac{(t - t_1)(t - t_2)}{(t_0 - t_1)(t_0 - t_2)} + y_1 \cdot \frac{(t - t_0)(t - t_2)}{(t_1 - t_0)(t_1 - t_2)} + y_2 \cdot \frac{(t - t_0)(t - t_1)}{(t_2 - t_0)(t_2 - t_1)}.$$

Choosing the function values $(t_0, y_0) = (7, 10500)$, $(t_1, y_1) = (8, 10500)$ and $(t_2, y_2) = (9, 10000)$ yields

$$p_2(t) = 10500 \cdot \frac{(t - 8)(t - 9)}{(-1) \cdot (-2)} + 10500 \cdot \frac{(t - 7)(t - 9)}{1 \cdot (-1)} + 10000 \cdot \frac{(t - 7)(t - 8)}{2 \cdot 1}.$$

Some simplification gives

$$p_2(t) = 5250 \cdot (t^2 - 17t + 72) - 10500 \cdot (t^2 - 16t + 63) + 5000 \cdot (t^2 - 15t + 56)$$

and finally

$$p_2(t) = 250 \cdot (-t^2 + 15t - 14).$$

MATLAB-code for the Lagrange polynomial evaluation:

```
clear all
close all

% All the given datapoints
x = 1:10;
y = [10600 11000 10400 9800 10800 11000 ...
     10500 10500 10000 8800];
plot(x, y, 'o')
hold on

% The coefficients of the polynomial.
% See help for function polyval for the order.
p2 = [-250 3750 -3500];

% Evaluate the polynomial in January and April.
y_january = polyval(p2, 1)
y_april    = polyval(p2, 4)

xx = 0:0.1:12;
yy = polyval(p2, xx);
plot(xx, yy)
title('Dow Jones index 2001 with Lagrange interpolation')
xlabel('Month')
ylabel('Indexvalue')
```

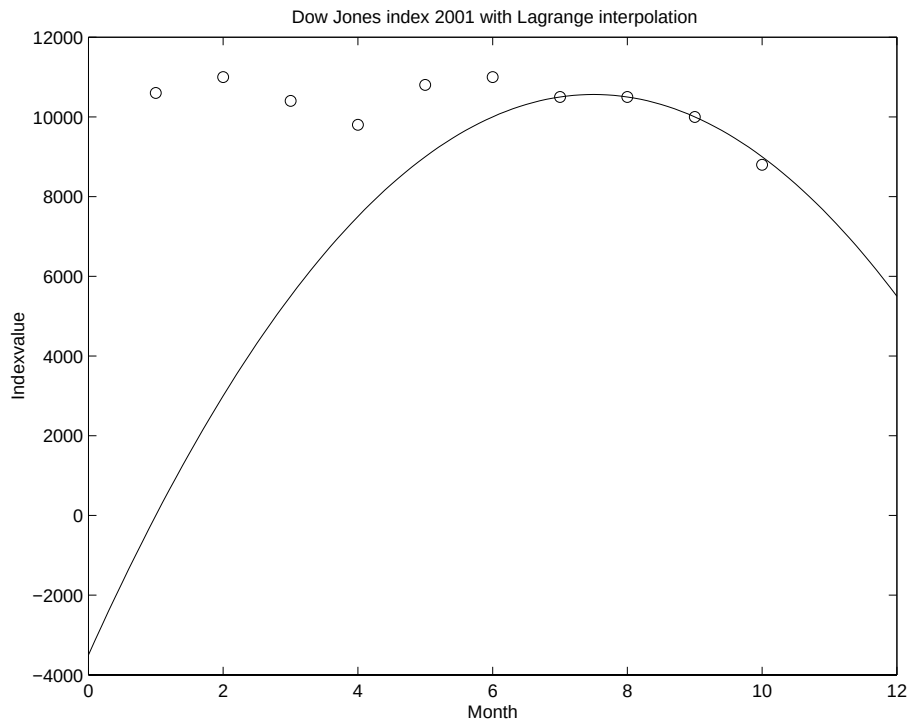


Figure 3: Dow Jones index 2001 interpolated by a Lagrange polynomial.

As can be seen from figure 3, it is impossible to extrapolate values from the polynomial. The polynomial values are not realistic outside the interval for which the polynomial was calculated. The estimated value in January is zero (should be 10600) and 7500 in April (should be 9800).

Task 2b):

Interpolating ten data points result in a nine degree polynomial. The MATLAB-algorithm is

```
figure
p = polyfit(x,y,9);
xx = 1:0.1:12;
yy = polyval(p,xx);
plot(xx,yy,x,y,'o')
axis([0 12 0 12000])
title('Dow Jones index 2001 with polynomial interpolation')
xlabel('Month')
ylabel('Indexvalue')
```

This figure (4) shows that when extrapolating, the result is just as bad as for Lagrange interpolation. Also, the interpolating function is oscillating a

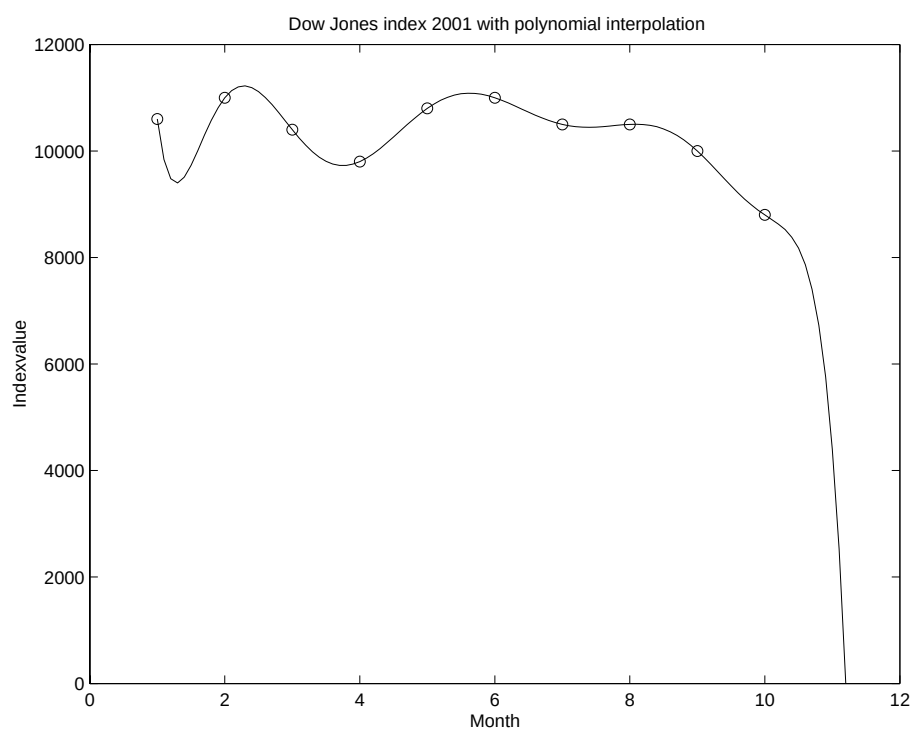


Figure 4: Down Jones index 2001 interpolated with a nine degree polynomial.

little too much between the data points in the beginning of the year (Runge's phenomenon) to be satisfactory.

3. Lagrange Polynomials.

Let us denominate $p(t) = \sum_{i=0}^n a_i L_i^n(t)$ for a set of fixed but arbitrary points $\{t_0, t_1, t_2, \dots, t_n\} \subset [a, b]$. Now, recalling that $L_i^n(t_j) = \delta_{ij}$ we obtain that

$$p(t_j) = \sum_{i=0}^n a_i L_i^n(t_j) = a_j L_j^n(t_j) = a_j.$$

Hence, if we found a sequence of coefficients a_i such that $p(t) = 0 \ \forall t \in [a, b]$ then in particular for $t = t_j$ we have that $p(t_j)$ has also to be equal to zero, and as a consequence $a_j = 0$. The reasoning follows for every $j = 0, 1, 2, \dots, n$.